SignBridge

Hackathon project guide · American Sign Language

Project overview

SignBridge turns typed or spoken English into a visual ASL reference. A user can review the text, look up a sign, play a recorded signer clip when one is available, and download the written notation. The project explores how these tools can make it easier to discover and share signs.

Sutton SignWriting uses symbols to describe visible features of signing, such as handshape, orientation, and movement. SignBridge displays that notation alongside a signer clip for dictionary entries. The notation and clips are drawn from independent sources and can show different variants of a sign.

What the prototype supports

- **Dictionary lookup.** The included entries are Thank you, Please, Goodbye, Hello, Yes, Water, Eat, Good, and What. Common aliases such as “Hi” are supported. Results can include a signer clip, notation, and an SVG download.
- **Gemini intent matching.** When configured, Gemini can match a paraphrase such as “I appreciate it” to the existing Thank you entry. It selects from the dictionary.
- **Experimental sentence generation.** A separate English-to-ASL model produces Formal SignWriting for input outside the dictionary. The browser renders the symbols and offers an .fsw download. Generated results may contain ASL errors and have no signer clip.
- **Voice interaction.** The microphone can fill in the text field in a supported browser. The user reviews the transcript before translating. The app can also speak a confirmation.

Demo in three minutes

1. Open the app in desktop Chrome. Click **Record Voice**, say **“Thank you”**, review the transcript, and select **Translate**. Type it if microphone access is unavailable.
2. Play the signer clip and download the SignWriting SVG. With Gemini enabled, try **“I appreciate it”** to demonstrate intent matching.
3. In the local app with the model running, enter **“I like green apples.”** Show the experimental symbols and download the Formal SignWriting file.

Public dictionary demo: SignBridge on Netlify. Use the local setup below for the sentence generation demo.

How it works

The React and TypeScript interface runs with Vite. It accepts text or a browser-generated transcript and displays translation results, notation, media playback, and export controls.

1. The app checks the bundled dictionary and can call the Netlify `translate` function for Gemini intent matching. The function limits Gemini to the supported dictionary IDs. Exact dictionary matches also work when Gemini is unavailable.
2. If there is no dictionary match, the app calls the Netlify `generate` function. That function sends the English input to the separate Python inference service.
3. The Python service uses a pretrained Sockeye checkpoint for English-to-ASL SignWriting. It returns Formal SignWriting, a text encoding for the notation. The frontend validates it and loads the corresponding SVGs from SignPuddle.

Repository map

- `src/` contains the interface, voice controls, dictionary, and notation renderer.
- `netlify/functions/` contains the server-side Gemini and model endpoints.
- `model/` contains the Python inference service, Dockerfile, and model setup instructions.
- `public/` contains bundled notation, signer clips, and media attribution.
- `netlify.toml` defines the web build and functions deployment.

Data and limitations

Browser speech recognition may use an online service. With Gemini enabled, the submitted text is sent to Google for intent matching. The model service receives unmatched text for inference, and SignPuddle receives the resulting notation for rendering. Credentials are configured server-side.

The dictionary has a fixed set of entries. Sentence generation is experimental and should be reviewed by fluent ASL users before it is used for important communication. Recorded clips are provided by ASL Signbank under CC BY-NC-SA 4.0. The Sockeye checkpoint is available under CC BY-NC 4.0. See media and model credits (public/credits.txt) for the source records and license terms.

Run and share

Start the web app

Install Node.js 22.12 or newer. In the project folder, run:

```
npm ci
npm run dev
```

Open the address Vite prints. Try **“Water”** to confirm that the dictionary, notation, and signer clip load. To enable Gemini, copy .env.example to .env, set your own GEMINI_API_KEY, and restart Vite.

Start sentence generation

With Docker running, use a second terminal in the project folder:

```
docker build -t signbridge-model model
docker run --rm -p 127.0.0.1:8080:7860 signbridge-model
```

The build downloads the model checkpoint. For a Python 3.11 setup or a direct model smoke test, follow the model service guide (model/README.md).

GitHub and Netlify handoff

Share the source, lockfile, bundled assets, .env.example, and documentation. Exclude .env, .netlify/, node_modules/, dist/, and virtual environments. A GitHub upload shares the code. Deploying updates from that repository requires connecting it to the existing Netlify site or deploying from a local checkout.

The existing Netlify demo serves dictionary lookup and Gemini matching. To publish sentence generation, deploy the Python service to a separate HTTPS host, set SIGNWRITING_MODEL_URL in Netlify, and redeploy. If the model host uses bearer authentication, configure SIGNWRITING_MODEL_TOKEN on both sides. See the project README (README.md) for deployment commands.